36. Kod Kalite Metrikleri: SonarQube, Linter'lar ve Statik Analiz

Teknik Borç Faizini Ödemeden Sistemi Sürdürülebilir Kılmak
Bölüm 6 • Zorluk Derecesi: Orta / İleri Seviye • Tahmini Okuma Süresi: 13 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Teknik borç (Technical Debt) nedir ve bir startup'ın ilk 6 ayında hızlı olmak için aldığı borçlar 2. yılda şirketi nasıl iflasa sürükler?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 Teknik borç, hızlı teslimat yapmak için kaliteden ödün vermektir; zamanında ödenmezse faizi tüm ekibi felç eder. - 📌 Statik analiz (ESLint, Biome, SonarQube), kodu çalıştırmadan potansiyel güvenlik açıklarını ve karmaşıklığı tespit eder. - 📌 Siklomotik Karmaşıklık (Cyclomatic Complexity), bir fonksiyondaki karar yollarının sayısıdır; 10'un üzerinde olması tehlike çanıdır.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Kredi kartıyla alışveriş yaptığınızı düşünün.

Kredi kartı harika bir kolaylıktır: Paranız yokken bile istediğiniz ayakkabıyı alırsınız (Hızlı kod yazıp teslim etmek). Ancak her ay asgari tutarı bile ödemezseniz ne olur? Faiz biner, faizin faizi biner ve sonunda haciz gelir!

Yazılımda **Teknik Borç (Technical Debt)** işte o kredi kartıdır: - "Şimdilik test yazmayalım, sonra bakarız." - "Buradaki if-else'i geçici yazdım, haftaya düzeltirim." - "Kütüphane deprecated oldu ama şimdilik çalışıyor."

1 yıl sonra bir butonu değiştirmek 3 hafta sürer hale gelir çünkü kimse o borç batağına dokunamaz!

Statik analiz ve linter araçları, her gün borcunuzu ödemenizi hatırlatan akıllı finans danışmanınızdır.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Teknik Borç ve Kod Kalite Piramidi:

```
[KOD YAZILIRKEN]
       |
       ▼
[1. Linter & Formatlayıcı (Biome / ESLint / Prettier)]
   └── Söz dizimi hatalarını, kullanılmayan değişkenleri anında yakalar.
       |
       ▼
[2. Tip Kontrolü (TypeScript / MyPy)]
   └── Tip güvenliğini ve şema uyumunu garanti eder.
       |
       ▼
[3. Karmaşıklık ve Koku Analizi (SonarQube / Code Climate)]
   └── Siklomotik karmaşıklığı, mükerrer kodları (duplicate) ölçer.
       |
       ▼
[TEMİZ & SÜRDÜRÜLEBİLİR ÜRÜN]
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Kod Kalitesi ve Statik Analiz Uzmanı
Aşağıdaki yüksek karmaşıklığa sahip (Cyclomatic Complexity > 12) JavaScript
fonksiyonunu incele.

## KOD:
```typescript
function calculateUserTariff(user: any) {
  let rate = 10;
  if (user.isStudent) {
    if (user.age < 25) {
      rate -= 3;
    } else {
      rate -= 1;
    }
  } else if (user.isSenior) {
    if (user.hasCard) {
      rate -= 4;
    } else {
      rate -= 2;
    }
  } else {
    if (user.isVip) {
      rate -= 5;
    }
  }
  return rate;
}
```

GÖREV:

1. Bu koddaki Siklomotik Karmaşıklığı (Cyclomatic Complexity) analiz et.
2. 'Lookup Table' veya 'Kural Motoru (Rule Engine)' deseni kullanarak karmaşıklığı 2'nin altına düşüren temiz refactor'ü yap.
3. ESLint için karmaşıklık sınırını zorunlu kılan kural önerisini sun.
```

```
---

## 💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

#### ❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):
```typescript
// Yüksek Siklomotik Karmaşıklık: 6 adet iç içe if-else ve 'any' kullanımı
export function calculateTariffNaive(user: any): number {
  if (user.isStudent && user.age < 25) return 7;
  if (user.isStudent && user.age >= 25) return 9;
  if (user.isSenior && user.hasCard) return 6;
  if (user.isSenior && !user.hasCard) return 8;
  if (user.isVip) return 5;
  return 10;
}
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```typescript
// Deklaratif ve Sıfır Karmaşıklık: Kural Listesi Deseni
export interface TariffUser {
  isStudent?: boolean;
  age?: number;
  isSenior?: boolean;
  hasCard?: boolean;
  isVip?: boolean;
}

type TariffRule = {
  predicate: (user: TariffUser) => boolean;
  discount: number;
};

const TARIFF_RULES: readonly TariffRule[] = [
  { predicate: (u) => Boolean(u.isVip), discount: 5 },
  { predicate: (u) => Boolean(u.isSenior && u.hasCard), discount: 4 },
  { predicate: (u) => Boolean(u.isStudent && (u.age ?? 0) < 25), discount: 3 },
  { predicate: (u) => Boolean(u.isSenior && !u.hasCard), discount: 2 },
  { predicate: (u) => Boolean(u.isStudent && (u.age ?? 0) >= 25), discount: 1 },
];

const BASE_RATE = 10;

export function calculateUserTariff(user: TariffUser): number {
  const matchedRule = TARIFF_RULES.find((rule) => rule.predicate(user));
  return BASE_RATE - (matchedRule?.discount ?? 0);
}
```

💡 **Kod Analizi:** İç içe if-else blokları tamamen ortadan kaldırılarak deklaratif bir kural listesine (`TARIFF_RULES`) dönüştürülmüştür. Fonksiyonun karmaşıklığı 1'e düşürülmüş ve yeni bir tarife kuralı eklemek tek bir satır eklemeye indirgenmiştir.
```

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Linter uyarılarını (warnings) önemsemeyip yüzlerce uyarının birikmesine izin vermek.
  - **Çözüm:** CI pipeline'ında uyarıları hata kabul eden kuralı (`--max-warnings 0`) devreye alın.
- **Tuzak:** Teknik borcu tamamen görmezden gelip her sprintte sadece yeni özellik yazmaya çalışmak.
  - **Çözüm:** Her sprint kapasitenizin en az %15-20'sini teknik borç temizliğine ve refactor'e ayırın.
- **Tuzak:** Linter kurallarını çok katı yapıp ekibi kod yazamaz hale getirmek.
  - **Çözüm:** Pragmatik olun; kuralları takım kararıyla belirleyin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 36: Kod Kalite Metrikleri - Teknik Borcu Kontrol Altına Almak*

- 🖥️ **Slayt 1:** Teknik Borç (Technical Debt) nedir ve bileşik faizi nasıl işler?
- 🖥️ **Slayt 2:** Siklomotik Karmaşıklık (Cyclomatic Complexity) nasıl ölçülür?
- 🖥️ **Slayt 3:** Biome ve ESLint ile otomatik kod hijyeni kurmak.
- 🖥️ **Slayt 4:** Bölüm 6 Özeti: Test Odaklı Geliştirme ve Kalite Güvencesi.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Karmaşık iç içe if-else içeren bir dosyada ESLint karmaşıklık eklentisini çalıştırıp uyarının yandığını gösterin. Kural tablosuna refactor ettikten sonra uyarının söndüğünü canlı gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizde `eslint` veya `biome` çalıştırın. En karmaşık veya en çok uyarı veren dosyayı tespit edip en az 2 uyarısını temizleyin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Yazılım mühendisliğinde 'Siklomotik Karmaşıklık' (Cyclomatic Complexity) neyi ölçer? - A) Kodun kaç megabayt yer kapladığını - B) Bir fonksiyondaki bağımsız kod yürütme yollarının (if, else, while vb. dallanmaların) sayısını - C) Bilgisayarın ekran parlaklığını - D) GitHub'daki yıldız sayısını > **Doğru Yanıt:** B > **Açıklama:** Siklomotik karmaşıklık, koddaki mantıksal dallanma sayısını ölçer. Sayı ne kadar yüksekse kodu test etmek ve anlamak o kadar zorlaşır.

Soru 2: Teknik borcun (Technical Debt) zamanında ödenmemesinin ekipler üzerindeki en yıkıcı sonucu nedir? - A) Elektrik faturasının artması - B) Basit bir değişikliğin bile haftalar sürmesi, her güncellemede başka yerlerin kırılması ve geliştirici üretkenliğinin sıfıra yaklaşması - C) Sadece Python'da hata çıkması - D) Monitörün kapanması > **Doğru Yanıt:** B > **Açıklama:** Ödenmeyen teknik borç, kod tabanını kırıntı haline getirir; yeni özellik ekleme hızı çöker ve şirket teknik iflasa sürüklenir.